

Objective: Build a Full-Stack AI Fantasy Sports Web App.

Tech Stack:

- **Frontend:** React (Next.js) with Tailwind CSS + shadcn/ui.
- **Backend:** Python (FastAPI).
- **Database:** PostgreSQL (Replit Managed DB).
- **AI:** Groq SDK ([llama-3.1-8b-instant](#)).

Core Architecture:

1. **Sync Engine:** Implement a "Manual Sync" route ([/api/sync-espn](#)) that accepts [espn_s2](#) and [SWID](#) tokens.
2. **User Experience:** Build a "Sync Assistant" page that includes a copy-to-clipboard button for the Bookmarklet payload (provided below) and a visual guide.
3. **AI Trade Lab:** A trade evaluator that pulls roster data from the database, sends the context to Groq, and returns a JSON object (Win Score A/B + Analysis).
4. **Database Strategy:** All ESPN data is cached in PostgreSQL. AI responses are cached in [AI_Cache](#) using the prompt hash to prevent rate-limit depletion.

Bookmarklet Payload (The iOS Sync Hack): *Create a utility on the Sync page that provides this code for users to save as a bookmark in Safari:* `javascript:(function(){const s2=document.cookie.match(/espn_s2=([^;]+)/)?.[1];const swid=document.cookie.match(/SWID=([^;]+)/)?.[1];if(s2&&swid){fetch('https://YOUR_REPLIT_URL/api/sync-espn',{method:'POST',headers:{'Content-Type':'application/json'},body:JSON.stringify({s2,swid})}).then(()=>alert('Sync Complete!'))}else{alert('Log into ESPN first!')}}());`

Step-by-Step Build Plan:

1. **Phase 1:** Setup the project, DB schema (Users, Leagues, Teams, Players), and FastAPI skeleton.
2. **Phase 2:** Implement the [/api/sync-espn](#) endpoint using [espn-api](#).
3. **Phase 3:** Build the Sync Assistant UI with the bookmarklet generator.
4. **Phase 4:** Build the AI Trade Lab logic (Backend service for Groq calls + React frontend UI).
5. **Phase 5:** Implement Caching Layer to ensure we don't exceed Groq's 30 RPM limit.